

激光 SLAM 建图实验指导

1. 实验描述

构建地图环境并准确定位是机器人运动规划的重要前提。酒店应用场景下的单层搜索实际上是 SLAM 功能与障碍物检测过程。SLAM 功能通过机器人定位与地图构建的实现与配合，奠定机器人运动规划的基础，而障碍物检测为机器人运动中的实时调整又增加了可靠性的保障。

本次实验基于真实应用场景，采用两种建图的算法，通过遥控手柄的控制方式去操控智能机器人在应用场景中完成地图的构建与自身位置的确定，同时能检测出地图中的障碍物，最终实现对酒店单楼层的搜索任务，即环境建模。

2. 实验目标

- (1) 理解 Gmapping 与 Hector 两种建图算法的相关参数配置；
- (2) 掌握使用 Gmapping 与 Hector 两种算法进行建图的相关方法与步骤；
- (3) 掌握地图保存的方法，并对建图结果进行分析评估。

3. 实验步骤

智能机器人分别使用了 Gmapping 算法和 Hector 算法进行 SLAM 操作，下面我们分别学习其参数配置、建图操作及地图保存和结果分析。

步骤 1: Gmapping 算法参数配置

Gmapping 算法可以从激光雷达、里程计等传感器数据中实时地构建二维地图，算法内容包括四个主要部分：激光雷达驱动程序、激光数据滤波器、扫描匹配算法、粒子滤波器。它的主要流程是：首先，激光雷达驱动程序负责接收激光雷达的数据，并将数据转换为 ROS 接受的格式；然后，激光数据滤波器用于过滤实际环境中的杂乱数据，以确保算法的精度和稳定性；接下来，扫描匹配算法用于确定机器人的位姿；最后，粒子滤波器用于对机器人的位姿估计进行变化，并利用估计的位姿信息对地图进行更新。

Gmapping 算法是基于粒子滤波的激光 SLAM 算法，其工作过程严重依赖里程计。因为它具有回环检测功能，可以构建大型的、连续的地图，故而适用于大型的、复杂的场景。然

而，对于无人机及地面不平坦的区域，Gmapping 可能无法适应。

ROS 系统可以直接安装 Gmapping 算法的功能包，然后进行调用。但是调用时还需要配置相关参数，以确保最终的建图结果。在智能机器人中已经配置好了相关算法，直接根据后续步骤进行学习和实操即可。

（1）功能启动的 launch 文件

打开工程文件夹，从文件夹 `src\sebot_slam\sebot_slam\launch` 中找到 Gmapping 算法建图功能的 launch 文件。

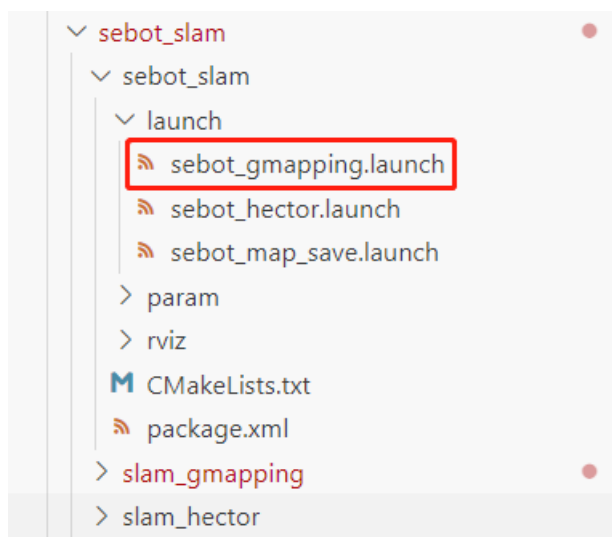


图 3-1 Gmapping 算法建图启动文件

此 launch 文件的功能较为简单，通过该文件可以同时启动多个节点，主要是启动建图功能中需要使用到的相关功能包及节点，包括启动机器人的控制节点包括超声波、里程计、IMU 使能、启动激光雷达、启动融合里程计、发布机器人静态坐标、启动遥控手柄控制节点、启动 rviz 可视化工具等，相关的命令关键词我们在前面也已经学习过，整个文件的相关功能可以直接结合注释进行理解和练习。

重点注意的是第 20 行的指令，此行指令表示的是引用 `gmapping.xml` 文件作为建图操作时的相关参数配置文件，注意其路径，此文件中包括了使用 Gmapping 算法建图时需要配置的各个参数，也是我们需要着重学习和分析的内容。

```

1 <launch>
2   <!-- 启动机器人的控制节点 -->
3   <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4     <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5     <param name="ultraEnale" type="bool" value="false"/> <!-- 超声波使能 -->
6     <param name="odomEnable" type="bool" value="true"/> <!-- 里程计使能 -->
7     <param name="imuEnable" type="bool" value="true"/> <!-- imu使能 -->
8   </node>
9
10  <!-- 启动激光雷达 -->
11  <include file = "${find rplidar_ros}/launch/rplidar.launch" />
12
13  <!-- 启动融合里程计 -->
14  <include file = "${find robot_pose_ekf}/launch/robot_pose_ekf.launch"/>
15
16  <!-- 发布机器人静态坐标 -->
17  <node pkg="sebot_robot" type="sebot_transform" name="sebot_transform" respawn="true"/>
18
19  <!-- 引用xml文件来启动gmapping算法 -->
20  <include file="${find sebot_slam}/param/gmapping.xml"/>
21
22  <!-- 启动遥控手柄控制节点 -->
23  <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
24
25  <!-- 启动rviz监视机器人建图 -->
26  <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d ${find sebot_slam}/rviz/slam.rviz"/>
27 </launch>

```

图 3-2 启动文件内容

(2) 配置相关参数

打开工程文件夹，从文件夹 `src\sebot_slam\sebot_slam\param` 中找到 Gmapping 算法配置的相关文件。

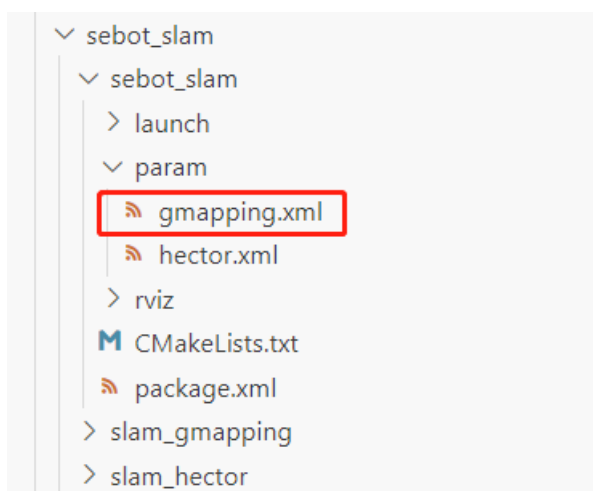


图 3-3 Gmapping 算法参数配置文件

文件中的主要内容都是对参数的设置，参数设置完成后都会传入到 Gmapping 算法的源码文件中，源码根据这些参数的设置进行计算并输出结果。

配置文件中主要的几个参数标签含义如下：

<arg>：该标签允许在命令行、多个文件中传递参数。

<param>: 该标签定义要在参数服务器上设置的参数。

<remap>: 节点重映射, 用于改变节点订阅或发布的话题。

配置文件中主要是设置了 Gmapping 算法节点的相关参数信息, 包含启动节点、话题发布、基础坐标系、雷达数据处理、地图参数设置等, 具体的参数作用结合 Gmapping 算法及代码注释自行理解即可。如果需要修改参数以观察不同结果时, 建议只修改<param>标签中的 value 的数值即可。配置文件的源码拆解阐述如下:

1、基础的坐标系和话题的设置: 包括 map_topic 地图发布话题、激光雷达话题、里程计及地图坐标系、机器人坐标系等。

```
<arg name="map_topic" default="map" /> <!-- 地图发布的话题名称 -->
<arg name="odom_frame" default="odom_combined" /> <!-- 里程计坐标系 -->

<!-- 启动 gmapping 算法的节点 -->
<node pkg="gmapping" type="slam_gmapping" name="slam_gmapping"
respawn="true">
  <!-- 坐标系名称 -->
  <remap from="map" to="$(arg map_topic)" /> <!-- 地图发布名称 -->
  <param name="odom_frame" value="$(arg odom_frame)" /> <!-- 里程计坐标系 -->
  <param name="map_frame" value="map" /> <!-- 地图坐标系 -->
  <remap from="scan" to="scan" /> <!-- 激光雷达话题名称 -->
  <param name="base_frame" value="base_footprint" /> <!-- 机器人的坐标系 -->
```

2、对雷达数据的处理参数设置, 激光雷达探测最大范围默认 80, 每次扫描跳过的光束数量默认 0, 可改为 1, 同时设置扫描匹配的迭代次数默认为 5。

```
<!-- 雷达数据处理设置 -->
<param name="maxRange" value="4"/> <!-- 传感器的最大范围 -->
<param name="maxUrange" value="3.5"/> <!-- 默认值 80, 激光可探测的最大范围, 探测的最大范围要小于传感器的真实最大范围 -->
<param name="lskip" value="0"/> <!-- 默认值 0, 每次扫描跳过的光束数量, 在计算压力过大时可以改成 1 -->
<param name="iterations" value="5"/> <!-- 默认值 5, 扫描匹配的迭代次数 -->
```

3、对地图参数的相关设置: 包括 x、y 向坐标、地图分辨率等, 可修改 value 的参数进行测试验证。

```
<!-- 地图参数设置 -->
<param name="xmin" value="-5.0"/> <!-- 默认值-100, 地图 x 向初始最小尺寸: m -->
```

```

<param name="ymin" value="-5.0"/> <!-- 默认值-100, 地图 y 向初始最小尺寸:
m -->
<param name="xmax" value="5.0" /> <!-- 默认值 100, 地图 x 向初始最大尺寸 -->
<param name="ymax" value="5.0" /> <!-- 默认值 100, 地图 y 向初始最大尺寸 -->
<param name="delta" value="0.03"/> <!-- 默认值 0.05, 地图的分辨率 (最小栅
格的边长) -->
<param name="srr" value="0.1"/> <!-- 运动模型的噪声参数 -->
<param name="srt" value="0.2"/>
<param name="str" value="0.1"/>
<param name="stt" value="0.2"/>

```

4、对算法中其他关键参数的设置：包括地图更新频率和时效性设置、极大似然估计、影响粒子滤波算法的精度和速度等设置。

```

<!-- 影响地图更新频率和时效性 -->
<param name="lstep" value="0.05"/> <!-- 默认值 0.05, 机器人采样时的线性
步长 -->
<param name="astep" value="0.05"/> <!-- 默认值 0.05, 机器人采样时的角度
步长 -->
<param name="linearUpdate" value="0.5"/> <!-- 默认值 1, 每移动此长度后进
行地图的更新: m -->
<param name="angularUpdate" value="0.5"/> <!-- 默认值 0.5, 每旋转此角度
后进行地图的更新: rad -->
<param name="map_update_interval" value="3.0"/> <!-- 默认值 5.0, 地图更
新频率 (s), 该值越低, 更新频率越快且计算负载越大 -->
<param name="minimumScore" value="80"/> <!-- 默认值 0.0, 激光的置信度,
越高说明对激光匹配算法的要求越高, 激光的匹配也越容易失败而转去使用里程计数据, 而设
的太低又会使地图中出现大量噪声, 只有扫描匹配成功的数据才能更新地图 -->
<param name="temporalUpdate" value="4.0"/> <!-- 默认值-1.0,
当最新扫描处理的速度比更新的速度慢时立即进行一次扫描匹配 (为负数时关闭该功能) -->

<!-- 极大似然估计 -->
<param name="sigma" value="0.05"/> <!-- endpoint 匹配标准差 -->
<param name="lsigma" value="0.075"/> <!-- 默认值 0.075, 用于扫描匹配概率
的激光标准差 -->
<param name="kernelSize" value="1"/> <!-- 默认值 1, 地图的核的大小 (小范
围时核越小越好, 大范围时核越大越好 -->
<param name="ogain" value="3.0"/> <!-- 默认值 3.0, 似然估计为平滑重采样影
响使用的增益 -->
<param name="llsamplerange" value="0.01"/> <!-- 默认值 0.01, 似然计算的
平移采样距离 -->
<param name="llsamplestep" value="0.01"/> <!-- 默认值 0.01, 似然计算的
平移采样步长 -->
<param name="lasamplerange" value="0.005"/> <!-- 默认值 0.005, 似然计算
的角度采样距离 -->

```

```

<param name="lasamplestep" value="0.005"/> <!-- 默认值 0.005, 似然计算
的角度采样步长 -->

<!-- 影响粒子滤波算法的精度和速度 -->
<param name="resampleThreshold" value="0.5"/> <!-- 默认值 0.5, 基于 Neff
的重采样阈值 -->
<param name="particles" value="30"/> <!-- 默认值 30, 滤波器中的粒子数
目 -->
<param name="transform_publish_period" value="0.05" /><!-- 多长时间发
布一次 tf 转换 (map->odom 转换), 单位是秒 -->
<param name="occ_thresh" value="0.25" /> <!-- 占有栅格地图的阈值, 只有当
前单元格的占有率超过该值才认为是被占有。如果设置该值为 1 的话就会导致任何障碍物都不
会认为是占有 -->
<param name="throttle_scans" value="1" />

</node>

```

将以上 Gmapping 算法的各项参数进行合理配置后, 才能根据这些参数进行计算从而实现建图功能, 最终获得理想的建图效果。如果建图效果不佳, 可以对参数文件内的参数信息进行适当调整。在源码文件中, 课程已经设置了默认的各种参数, 可直接开展实操, 但同学们可以修改多个参数, 结合现场的实际条件进行实操验证。

为了防止对初始配置造成错误配置后无法复原, 建议先在 param 文件夹下复制并重命名 gmapping.xml 文件, 接着根据需要对文件中的各项参数进行设置, 设置完成后保存文件。然后在 launch 文件夹下的 sebot_gmapping.launch 文件中修改对应的配置文件路径, 以此便可以在不影响原有工程效果的基础上充分进行测试和学习。

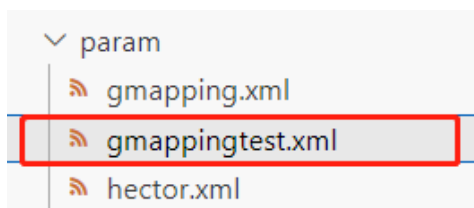


图 3-4 复制并重命名 Gmapping 算法参数配置文件

```

: > sebot_slam > sebot_slam > launch > sebot_gmapping.launch
1  <launch>
2    <!-- 启动机器人的控制节点 -->
3    <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4      <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5      <param name="ultraEnable" type="bool" value="false"/>      <!-- 超声波使能 -->
6      <param name="odomEnable" type="bool" value="true"/>      <!-- 里程计使能 -->
7      <param name="imuEnable" type="bool" value="true"/>      <!-- imu使能 -->
8    </node>
9
10   <!-- 启动激光雷达 -->
11   <include file = "${find rplidar_ros}/launch/rplidar.launch" />
12
13   <!-- 启动融合里程计 -->
14   <include file = "${find robot_pose_ekf}/launch/robot_pose_ekf.launch"/>
15
16   <!-- 发布机器人静态坐标 -->
17   <node pkg="sebot_robot" type="sebot_transform" name="sebot_transform" respawn="true"/>
18
19   <!-- 引用xml文件来启动gmapping算法 -->
20   <include file="${find sebot_slam}/param/gmappingtest.xml"/>
21
22   <!-- 启动遥控手柄控制节点 -->
23   <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
24
25   <!-- 启动rviz监视机器人建图 -->
26   <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d ${find sebot_slam}/rviz/slam.rviz"/>
27 </launch>

```

图 3-5 配置文件路径

步骤 2: Gmapping 算法建图实操

如果对 Gmapping 算法文件有修改，在步骤 1 的文件内容及相关路径设置完成后再进行实操。如果没有修改，可直接将机器人启动进行实操。

在机器人桌面打开终端，并输入以下命令即可开启 SLAM 建图功能。

```
roslaunch sebot_slam sebot_gmapping.launch
```

结合前面的代码逻辑可以理解，建图功能启动后会自动开启 Rviz 监视器，机器人桌面弹出 Rviz 可视化工具，Rviz 作用是查看机器人的各项数据，当在仿真环境或者实际环境中通过遥控手柄控制机器人行走时，Rviz 便会最大程度的采集场地障碍数据，同时在可视化界面中生成建图结果。

此时通过手柄控制机器人在真实场地内进行移动(在之前实训中已经学习了手柄控制的启动方法)，只需使用前进、后退、左转、右转以及速度设置的功能即可，建图的基本要求是操控机器人走遍地图的所有路径或角落，以最大程度获取完整的地图信息。

机器人走动过程中，同时观察 Rviz 软件界面内的地图形状，当地图与真实环境的形状一致时，即可进入下一步的地图保存操作。需要注意的是，建议实际场地为不规则形状，而非正方形等规则形状，这样在 Rviz 中容易看出地图构建是否完毕。

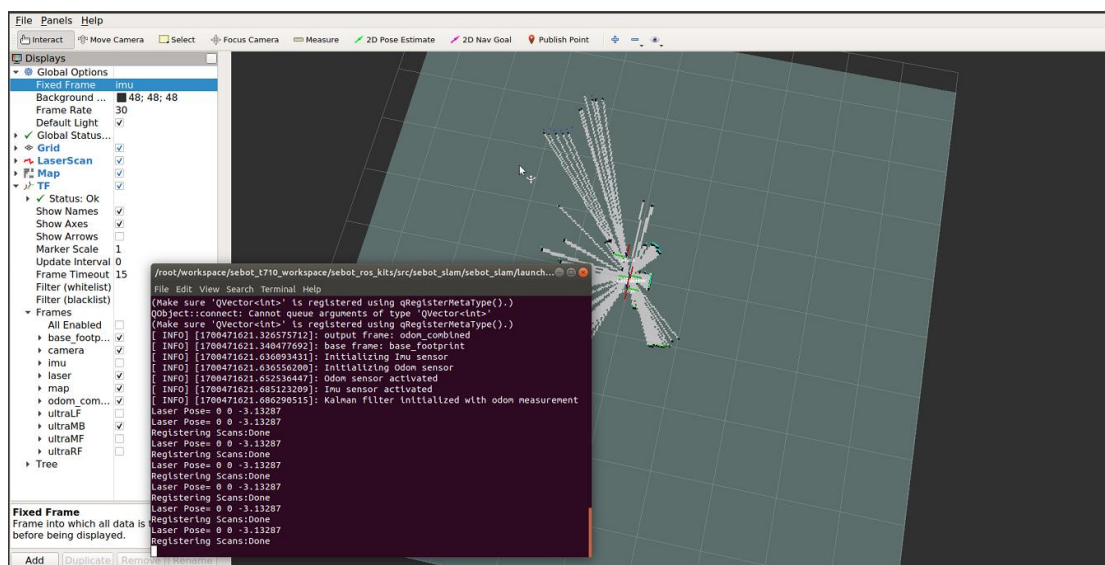


图 3-6 运行建图功能后的 Rviz 界面及终端

步骤 3: Gmapping 算法建图结果保存与分析

在步骤 2 中, 当 Rviz 中呈现地图与实际地图一致时, 可视为建图完成, 即可进行保存地图的操作。此操作是通过启动对应的 launch 文件 `src\sebot_slam\sebot_slam\launch\sebot_map_save.launch` 从而进行保存地图操作的。后续的 Hector 算法建图使用的保存命令也是此文件。

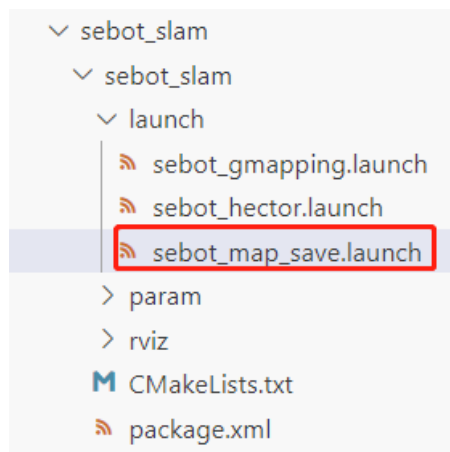


图 3-7 保存地图功能的启动文件

`sebot_map_save.launch` 文件中, `args` 后面的参数即保存后的地图文件路径及文件名。如果不做修改, 则会导致同名的新文件覆盖掉旧文件。因此实际操作时, 根据需要可以自行修改配置文件中地图文件的名称, 这样可以在不影响原有地图文件的条件下保存新的地图文件。例如想要分析不同参数对建图结果的影响时, 每次修改完算法的配置参数后还要修改建图后保存的地图文件名, 以便于保存不同的地图结果从而在后续分析时进行对比。

如下图操作即将地图名称进行了修改。

```
src > sebot_slam > sebot_slam > launch > sebot_map_save.launch
1 <!-- 保存地图 -->
2 <launch>
3   <!-- 启动仿真监控界面 -->
4   <node pkg="map_server" type="map_saver" name="map_server" args="-f $(find sebot_navigation)/map/map"/>
5 </launch>
```

图 3-8 修改前的地图保存路径

```
src > sebot_slam > sebot_slam > launch > sebot_map_save.launch
1 <!-- 保存地图 -->
2 <launch>
3   <!-- 启动仿真监控界面 -->
4   <node pkg="map_server" type="map_saver" name="map_server" args="-f $(find sebot_navigation)/map/map1"/>
5 </launch>
```

图 3-9 修改的地图保存路径

修改地图文件名及路径完成后保存 launch 文件，然后在机器人桌面打开新终端并输入以下命令即可保存建立完成的地图文件。

```
roslaunch sebot_slam sebot_map_save.launch
```

终端中运行完此程序后，在文件夹目录 `src\sebot_navigation\sebot_navigation\map` 下即可查看已保存的地图文件和配置文件。

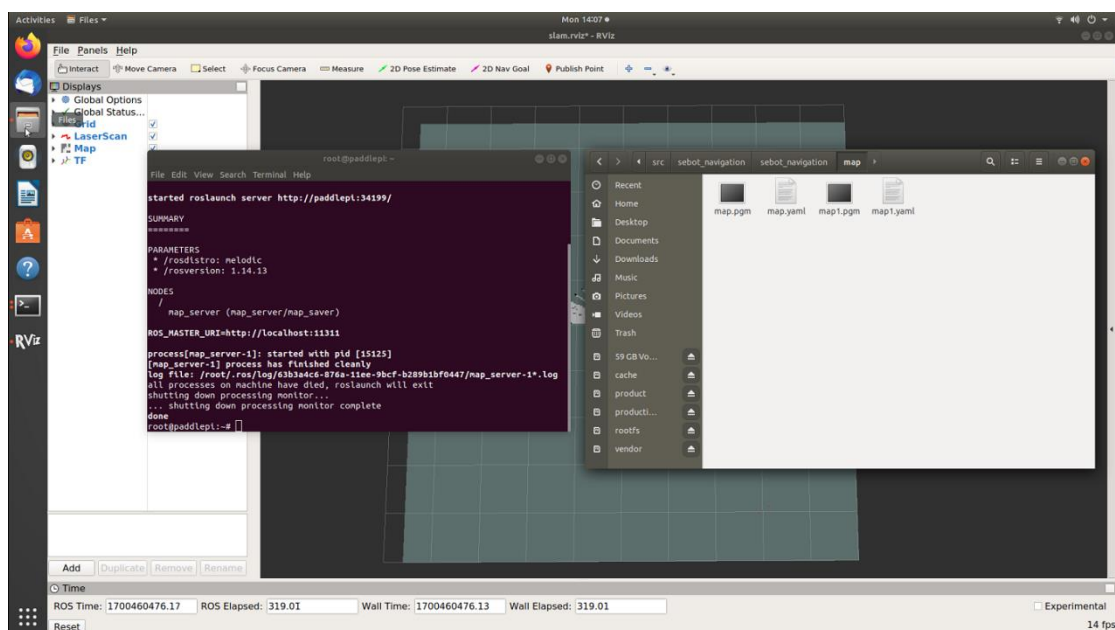


图 3-10 执行地图保存后的终端及地图保存的文件目录

打开保存的目录后，以 .pgm 为后缀的文件即地图图片，此格式为二进制灰度类型的编码文件，能够被常用的图片查看软件打开查阅，并且可以使用绘图软件进行二次编辑，例如瑕疵处理。

直接双击打开 `map.pgm` 即可查看，通过查看地图形状是否与真实场景相似即可判断建图结果的准确性。通常情况，建图的质量根据实际建图的场景而定，实验机器人实测在较为

复杂的室内结构、道路不平的区域中建图效果表现优良，但是推荐初学者在较为空旷和元素单一的室内环境再实施建图任务，可以更直观地理解参数作用及算法原理。

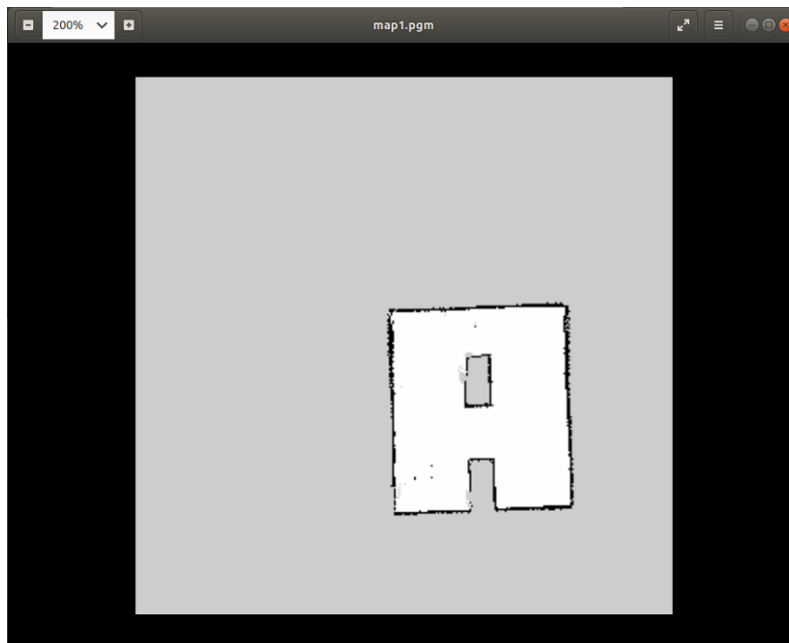


图 3-11 查看保存后的地图图片文件

以.yaml 为后缀的文件即地图配置文件。配置文件中相关参数说明如下：

image: 指定地图图像文件的路径，通常是 PNG 格式的栅格地图图像。

resolution: 指定每个栅格的物理尺寸。

origin: 定义地图原点在物理空间中的位置。

negate: 如果设置为 1，表示将栅格地图中的障碍物和自由空间取反。

occupied_thresh: 定义被视为障碍物的栅格状态阈值。

地图配置信息无需更改，但是如果要移动地图文件的话，需要将两者一起移动，保证始终处于同一文件夹下，以便后续操作中需要读取此地图的相关信息。

```
src > sebot_navigation > sebot_navigation > map > ! map1.yaml
1 image: /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_navigation/sebot_navigation/map/map.pgm
2 resolution: 0.003000
3 origin: [0.500000, 0.500000, 0.000000]
4 negate: 0
5 occupied_thresh: 0.65
6 free_thresh: 0.196
```

图 3-12 查看保存后的地图配置文件

步骤 4: Hector 算法参数配置

Hector 算法使用一种称为“粒子滤波器”的方法来估计机器人的位姿，并且通过优化的方式不断提高精度。它通过从激光雷达数据中提取特征点，并将这些特征点与预先存储的地图中的特征点进行匹配，来估计机器人的位姿。简单而言就是将激光点与已有的地图“对齐”，

即扫描匹配。扫描匹配就是使用当前帧与已经有的地图数据构建误差函数，使用高斯牛顿法得到最优解和偏差量。其工作是实现激光点到栅格地图的转换，如果某一时刻所有的激光点都能变换到栅格地图中，也就意味着匹配成功。Hector 算法使用这些位姿估计值来构建地图，并不断更新地图以反映机器人的运动。

Hector 算法基于优化算法，不需要里程计，但是对于雷达帧率要求很高，需要 40Hz 以上的雷达数据。Hector 算法可以适应空中或者地面不平坦的情况，因为它基于优化的方法，可以对机器人位姿进行直接估计。但是，Hector 没有回环检测功能，对于大型的、复杂的场景，可能会出现問題。

ROS 系统可以直接安装 Hector 算法的功能包，然后进行调用。但是调用时还需要配置相关参数，以确保最终的建图结果。在智能机器人中已经配置好了相关算法，直接根据后续步骤进行学习和实操即可。

（1）功能启动的 launch 文件

打开工程文件夹，从文件夹 `src\sebot_slam\sebot_slam\launch` 中找到 Hector 算法建图功能的 launch 文件。

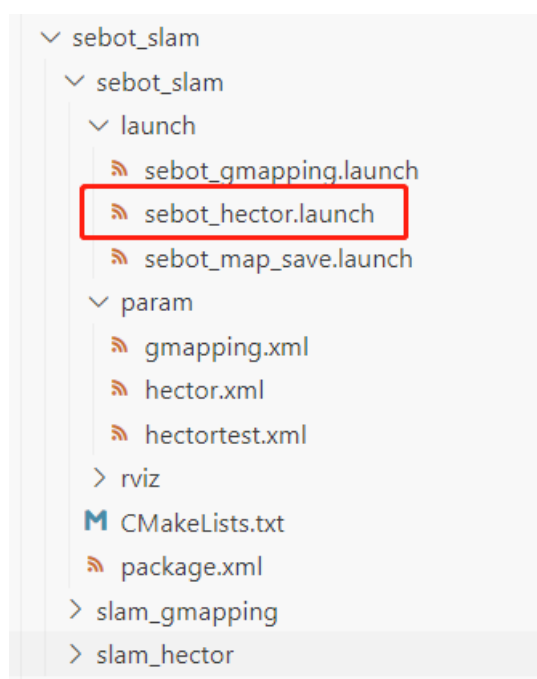


图 3-13 Hector 算法建图启动文件

此文件的功能较为简单，主要是启动 Hector 建图功能中需要使用到的相关功能包及节点。相比于前面的 Gmapping 算法，此配置中将里程计使能和 imu 使能均关闭，同时也没有启用里程计融合功能，其他部分则与 Gmapping 算法相同。重点是第 17 行的指令，此行指

令表示的是引用 `hector.xml` 文件作为建图操作的相关参数配置文件，此文件中包括了使用 Hector 算法建图时需要配置的几个参数，也是我们需要着重学习和分析的内容。

src > sebot_slam > sebot_slam > launch > sebot_hector.launch

```

1 <launch>
2 <!-- 启动机器人的控制节点 -->
3 <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4 <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5 <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
6 <param name="odomEnable" type="bool" value="false"/> <!-- 里程计使能 -->
7 <param name="imuEnable" type="bool" value="false"/> <!-- imu使能 -->
8 </node>
9
10 <!-- 启动激光雷达 -->
11 <include file = "${find rplidar_ros}/launch/rplidar.launch" />
12
13 <!-- 发布机器人静态坐标 -->
14 <include file="${find sebot_robot}/launch/sebot_transform.launch"/>
15
16 <!-- 引用xml文件来启动hector算法 -->
17 <include file="${find sebot_slam}/param/hector.xml"/>
18
19 <!-- 启动遥控手柄控制节点 -->
20 <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
21
22 <!-- 启动rviz监视机器人建图 -->
23 <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d ${find sebot_slam}/rviz/slam.rviz"/>
24
25 </launch>

```

图 3-14 启动文件内容

(2) 配置相关参数

打开工程文件夹，从文件夹 `src\sebot_slam\sebot_slam\param` 中找到 Hector 算法配置的相关文件。

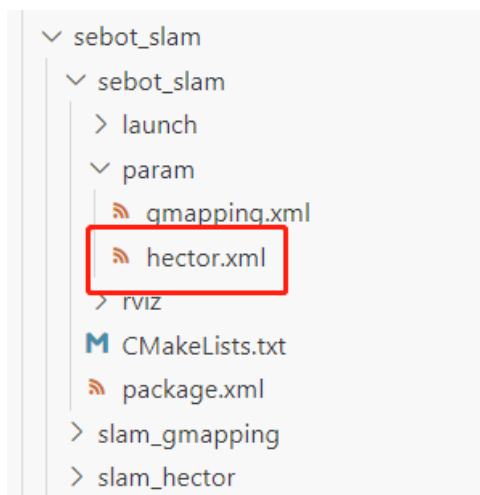


图 3-15 Hector 算法参数配置文件

文件中的主要内容都是对参数的设置，参数设置完成后都会传入到 Hector 算法的源码，源码根据这些参数的设置进行计算并输出结果。

配置文件中主要的几个参数标签含义与 Gmapping 算法中的说明相同。文件内容主要是设置了 Hector 算法节点的相关参数信息，其中包括基础的坐标系设置、TF 变换设置、地图

参数的相关设置、对雷达数据的处理以及话题发布的相关设置。此文件在运行时会调用到我们前面学习过的 TF 变换的功能程序。

具体的参数作用结合 Hector 算法及代码注释自行理解即可。如果需要修改参数以观察不同结果时，建议只修改<param>标签中的 value 的数值即可。配置文件的源码如下：

```
<launch>
  <arg name="map_topic" default="/map" /> <!-- 地图话题数据 -->
  <arg name="map_size" default="1024" /> <!-- 地图像素大小：例如 15x15m(分辨率 0.04), map_size=15/0.04=375 -->
  <arg name="use_tf_pose_start_estimate" default="false" />
  <arg name="map_scanmatch_frame_name" default="scanmatcher_frame"/>
  <!-- 发布 scanmatcher 到 map 转换的坐标名 -->

  <!-- 启动 hector 算法的节点 -->
  <node pkg="hector_mapping" type="hector_mapping"
name="hector_mapping">

    <!-- 坐标系名称 -->
    <remap from="map" to="$(arg map_topic)" /> <!-- 地图话题数据 -->
    <param name="map_frame" value="map" /> <!-- 地图坐标系 -->
    <param name="base_frame" value="base_footprint" /> <!-- 机器人基坐标系，用于定位和激光扫描数据的变换 -->
    <param name="odom_frame" value="base_footprint"/> <!-- 里程计坐标系 -->
    <param name="scan_topic" value="/scan"/> <!-- 设置激光数据的话题名称 -->

    <!-- Tf 设置 -->
    <param name="use_tf_scan_transformation" value="true"/> <!-- 启用激光雷达 TF 变换 -->
    <param name="use_tf_pose_start_estimate" value="$(arg use_tf_pose_start_estimate)"/>
    <param name="pub_map_odom_transform" value="true"/> <!-- 是否发布 map 与 odom 之间的坐标变换 -->

    <!-- 地图大小/起点设置 -->
    <param name="map_resolution" value="0.03"/> <!-- 默认值 0.025，地图的分辨率 -->
    <param name="map_multi_res_levels" value="2" /> <!-- 默认值 3，地图的多分辨率层数（影响地图精度） -->
    <param name="map_size" value="$(arg map_size)"/> <!-- 默认值 1024，正方形地图边长的栅格数目 -->
```

```

    <param name="map_start_x" value="0.5"/> <!-- 默认值 0.5 为居中, 在 x
    轴上, 地图坐标系的原点相对于栅格地图的位置 -->
    <param name="map_start_y" value="0.5" /> <!-- 默认值 0.5 为居中, 在
    y 轴上, 地图坐标系的原点相对于栅格地图的位置 -->

    <!-- 激光雷达设置 -->
    <param name="laser_min_dist" value="0.2" /> <!-- 默认值 0.4, 雷达
    扫描点的最小距离 (m), 即扫面点小于此值时被忽略 -->
    <param name="laser_max_dist" value="3.5" /> <!-- 默认值 30.0, 雷达
    扫描点的最大距离 (m), 即扫面点大于此值时被忽略 -->
    <param name="laser_z_min_value" value=" -1.0" /> <!-- 默认值-1.0,
    雷达扫描点相对于雷达坐标系的最小高度 (m), 即扫面点小于此值时被忽略 -->
    <param name="laser_z_max_value" value=" 1.0" /> <!-- 默认值 1.0, 雷
    达扫描点相对于雷达坐标系的最小高度 (m), 即扫面点大于此值时被忽略 -->

    <!-- 地图更新参数 -->
    <param name="map_update_distance_thresh" value="0.1"/> <!--
    默认值 0.4, 每移动此长度 (m) 后进行地图的更新 -->
    <param name="map_update_angle_thresh" value="0.2"/> <!--
    默认值 0.9, 每旋转此角度 (rad) 后进行地图的更新 -->
    <param name="update_factor_free" value="0.4"/> <!-- 默
    认值 0.4, 地图更新调整自由单元, 范围[0.0. 1.0], 0.5 意味着不改变 -->
    <param name="update_factor_occupied" value="0.9" /> <!-- 默
    认值 0.9, 地图更新调整占用单元, 范围[0.0. 1.0], 0.5 意味着不改变 -->

    <!-- 发布设置 -->
    <param name="map_pub_period" value="2.2"/> <!-- 默
    认值 2.0, map 发布的周期 (s) -->
    <param name="output_timing" value="false"/> <!-- 默
    认值 false, 决定是否通过 ROS_INFO 输出每次雷达扫描处理的时序信息 -->
    <param name="advertise_map_service" value="true"/> <!--
    是否发送地图话题-->
    <param name="scan_subscriber_queue_size" value="10"/> <!--
    默认值 5, scan 订阅的队列大小, 如果将日志文件以比实时速度更快的速度播放到
    hector_mapping, 则应置为较高的值(如 50) -->
    <param name="tf_map_scanmatch_transform_frame_name" value="$ (arg
    map_scanmatch_frame_name)" />

</node>
</launch>

```

为了防止对初始配置造成错误配置后无法复原, 建议先在 `param` 文件夹下复制并重命名 `hector.xml` 文件, 接着根据需要对文件中的各项参数进行设置, 设置完成后保存文件。然后在 `launch` 文件夹下的 `sebot_hector.launch` 文件中修改对应的配置文件路径, 以此便可以在

不影响原有工程效果的基础上充分进行测试和学习。

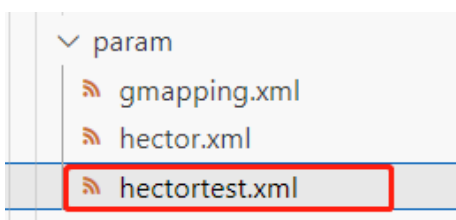


图 3-16 复制并重命名 Hector 算法参数配置文件

```
c > sebot_slam > sebot_slam > launch > sebot_hector.launch
1 <launch>
2 <!-- 启动机器人的控制节点 -->
3 <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4 <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5 <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
6 <param name="odomEnable" type="bool" value="false"/> <!-- 里程计使能 -->
7 <param name="imuEnable" type="bool" value="false"/> <!-- imu使能 -->
8 </node>
9
10 <!-- 启动激光雷达 -->
11 <include file = "$(find rplidar_ros)/launch/rplidar.launch" />
12
13 <!-- 发布机器人静态坐标 -->
14 <include file="$(find sebot_robot)/launch/sebot_transform.launch"/>
15
16 <!-- 引用xml文件来启动hector算法 -->
17 <include file="$(find sebot_slam)/param/hectortest.xml"/>
18
19 <!-- 启动遥控手柄控制节点 -->
20 <node pkg="sebot_robot" type="sebot_joyystick" name="sebot_joyystick" respawn="true"/>
21
22 <!-- 启动rviz监视机器人建图 -->
23 <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d $(find sebot_slam)/rviz/slam.rviz"/>
24
25 </launch>
```

图 3-17 配置文件路径

步骤 5: Hector 算法建图实操

如果对 Hector 算法文件有修改，在步骤 4 的文件内容及相关路径设置完成后再进行实操。如果没有修改，可直接将机器人启动进行实操。

根据步骤 4，参数配置文件内容及 launch 相关路径设置完成后，基于机器人打开终端并输入以下命令即可开启 Hector 建图功能。

```
roslaunch sebot_slam sebot_hector.launch
```

功能启动后会自动开启 Rviz 监视器，此时通过手柄控制机器人在真实场地内进行移动（只需使用前进、后退、左转、右转四个按钮即可），并观察 Rviz 软件界面内的地图形状，当地图与真实环境的形状一致时，即可进入下一步的地图保存操作。

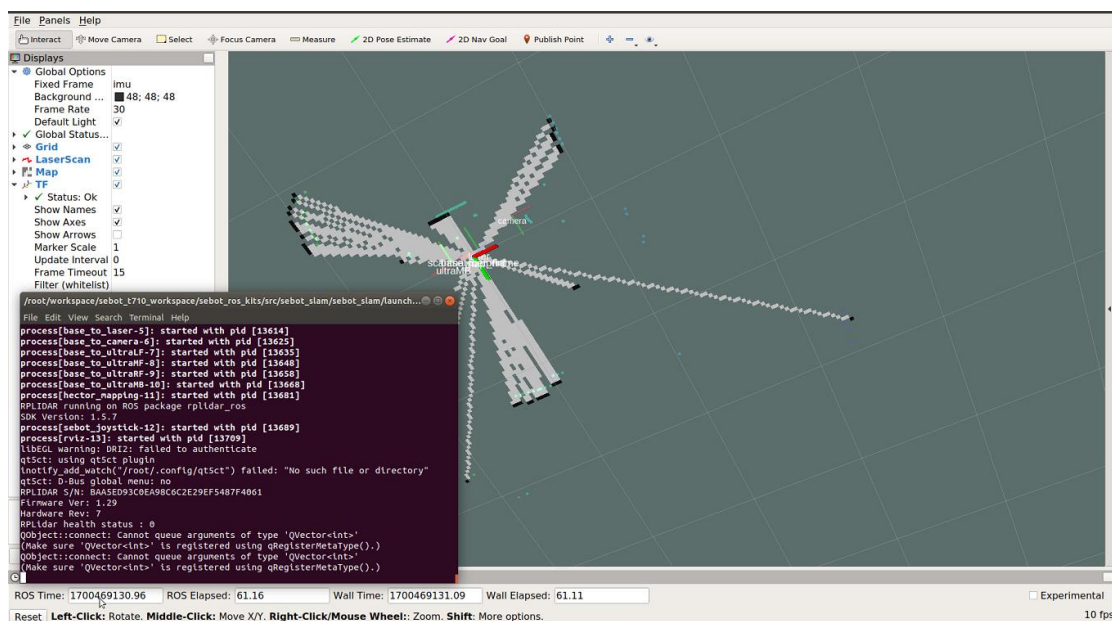


图 3-18 运行建图功能后的 Rviz 界面及终端

步骤 6: Hector 算法建图结果保存与分析

此步骤与 Gmapping 算法中保存地图的操作一致，下面直接根据图示进行操作和分析即可，详细说明可参考步骤 3。

首先在 launch 文件中修改地图文件名。

```
src > sebot_slam > sebot_slam > launch > sebot_map_save.launch
1 <!-- 保存地图 -->
2 <launch>
3 <!-- 启动仿真监控界面 -->
4 <node pkg="map_server" type="map_saver" name="map_server" args="-f $(find sebot_navigation)/map/map1"/>
5 </launch>
```

图 3-19 修改前的地图保存路径

```
src > sebot_slam > sebot_slam > launch > sebot_map_save.launch
1 <!-- 保存地图 -->
2 <launch>
3 <!-- 启动仿真监控界面 -->
4 <node pkg="map_server" type="map_saver" name="map_server" args="-f $(find sebot_navigation)/map/map2"/>
5 </launch>
```

图 3-20 修改后的地图保存路径

修改完成后保存 launch 文件，在机器人中打开新终端并输入以下命令即可保存建立完成的地图文件。

```
roslaunch sebot_slam sebot_map_save.launch
```

终端中运行完此程序后，在文件夹目录 src\sebot_navigation\sebot_navigation\map 下即可查看已保存的地图文件和配置文件。

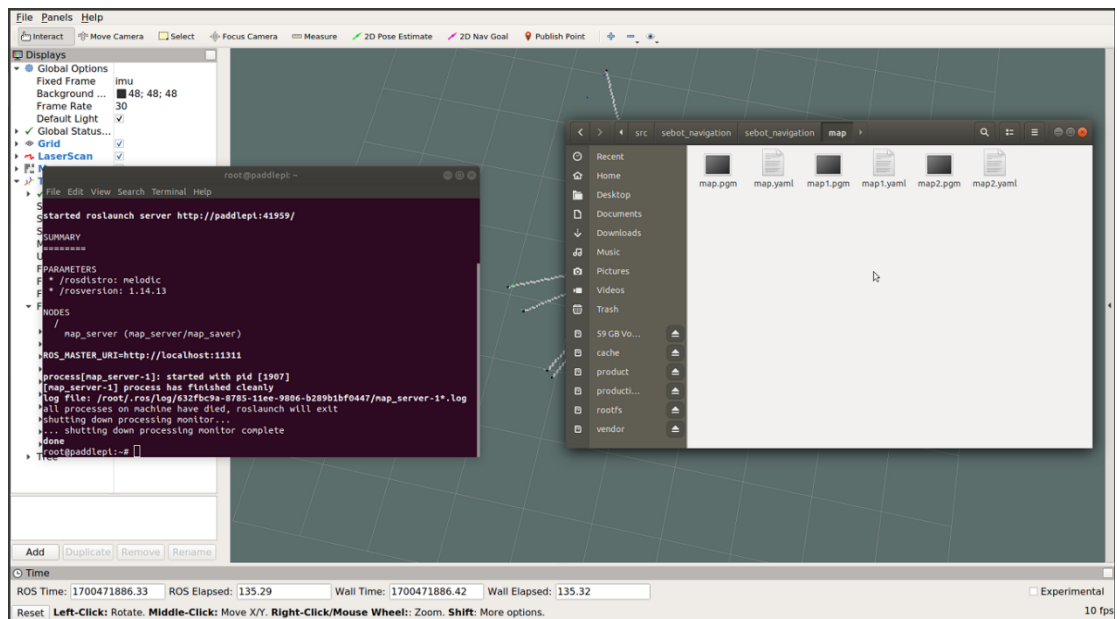


图 3-21 执行地图保存后的终端及地图保存的文件目录

打开地图文件后查看地图形状，进行建图结果分析。

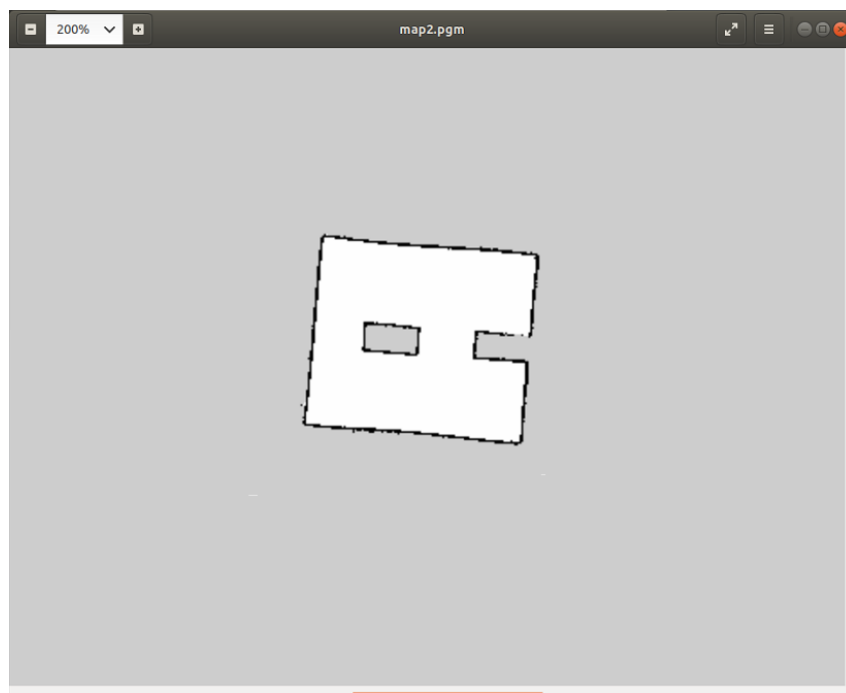


图 3-22 查看保存后的地图图片文件

相关配置文件内容如下：

```
src > sebot_navigation > sebot_navigation > map > ! map2.yaml
1 image: /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_navigation/sebot_navigation/map/map.pgm
2 resolution: 0.003000
3 origin: [0.500000, 0.500000, 0.000000]
4 negate: 0
5 occupied_thresh: 0.65
6 free_thresh: 0.196
```

图 3-23 查看保存后的地图配置文件

步骤 7：还原启动文件配置

前面在启动两种算法的配置文件时，都对文件内参数配置文件的路径进行了修改，建议在完成学习和测试后，将相关的配置文件路径还原，以免影响到其他功能的使用。

```
src > sebot_slam > sebot_slam > launch > sebot_gmapping.launch
1 <launch>
2 <!-- 启动机器人的控制节点 -->
3 <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4   <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5   <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
6   <param name="odomEnable" type="bool" value="true"/> <!-- 里程计使能 -->
7   <param name="imuEnable" type="bool" value="true"/> <!-- imu使能 -->
8 </node>
9
10 <!-- 启动激光雷达 -->
11 <include file = "$(find rplidar_ros)/launch/rplidar.launch" />
12
13 <!-- 启动融合里程计 -->
14 <include file = "$(find robot_pose_ekf)/launch/robot_pose_ekf.launch"/>
15
16 <!-- 发布机器人静态坐标 -->
17 <node pkg="sebot_robot" type="sebot_transform" name="sebot_transform" respawn="true"/>
18
19 <!-- 引用xml文件来启动gmapping算法 -->
20 <include file="$(find sebot_slam)/param/gmapping.xml"/>
21
22 <!-- 启动遥控手柄控制节点 -->
23 <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
24
25 <!-- 启动rviz监视机器人建图 -->
26 <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d $(find sebot_slam)/rviz/slam.rviz"/>
27 </launch>
```

图 3-24 还原 Gmapping 算法建图启动文件

```
src > sebot_slam > sebot_slam > launch > sebot_hector.launch
1 <launch>
2 <!-- 启动机器人的控制节点 -->
3 <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4   <!-- 是否发布传感器话题 value: true发布 false不发布 -->
5   <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
6   <param name="odomEnable" type="bool" value="false"/> <!-- 里程计使能 -->
7   <param name="imuEnable" type="bool" value="false"/> <!-- imu使能 -->
8 </node>
9
10 <!-- 启动激光雷达 -->
11 <include file = "$(find rplidar_ros)/launch/rplidar.launch" />
12
13 <!-- 发布机器人静态坐标 -->
14 <include file="$(find sebot_robot)/launch/sebot_transform.launch"/>
15
16 <!-- 引用xml文件来启动hector算法 -->
17 <include file="$(find sebot_slam)/param/hector.xml"/>
18
19 <!-- 启动遥控手柄控制节点 -->
20 <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
21
22 <!-- 启动rviz监视机器人建图 -->
23 <node pkg = "rviz" type = "rviz" name = "rviz" args = "-d $(find sebot_slam)/rviz/slam.rviz"/>
24
25 </launch>
```

图 3-25 还原 Hector 算法建图启动文件

自主定位与导航实验指导

1. 实验描述

在酒店服务场景中，机器人需要完成餐饮配送的任务：从起点运送餐饮物品到目标房间门口，由客人自行取走物品。实现该任务的技术要点即自主导航与路径规划技术。

自主导航与路径规划技术是实现机器人自主行驶的关键技术，主要涉及定位、建图、路径规划、避障等环节。此技术通常依赖于传感器数据，如激光雷达、摄像头等，来获取周围环境的信息，从而确定机器人的位置和姿态，接着导航系统可以根据这些信息来生成一条合适的路径，并控制机器人沿着这条路径前进。

针对场景任务，本节课我们基于激光导航技术，在实际场景中完成机器人自主导航与路径规划功能，通过这两种方式操控机器人完成餐饮配送任务。

2. 实验目标

- (1) 了解激光雷达导航技术的基本原理；
- (2) 理解智能机器人实现自主导航的功能逻辑；
- (3) 掌握自主导航的实现方法，验证其功能效果。

3. 实验步骤

步骤 1：理解自主导航原理及启动文件

自主导航功能中涉及了多个配置文件，通过不同配置文件间的参数传递，最终进行自主导航的计算过程。在进行学习时，首先需要理解多个配置文件间的逻辑关系，如果要进行参数修改和效果测试，还需要理解基本原理。随意改动文件内容可能会导致整个工程文件无法正常运行。

智能机器人的导航功能是基于已经保存好的地图文件进行的，使用前需要先确保地图形状与真实环境一致，否则路径规划无法正常进行。建议 SLAM 建图实验与本实验在同一场地中完成，本实验可直接调用上一实验构建完成的地图。

在 ROS 中，激光雷达导航主要分为两部分：一部分是定位功能，通过激光雷达传感器获取周围环境的点云数据，然后对这些数据进行处理和分析，以构建出机器人的周围环境地

图，从而确定机器人在当前地图中所处的位置；另外一部分是路径规划功能，此部分通过专门的算法进行计算，规划出从起点到终点的路线。通过这两部分功能的协作，便可以实现机器人的自主导航。

导航功能启动文件的路径 `src\sebot_navigation\sebot_navigation\launch` 如下。

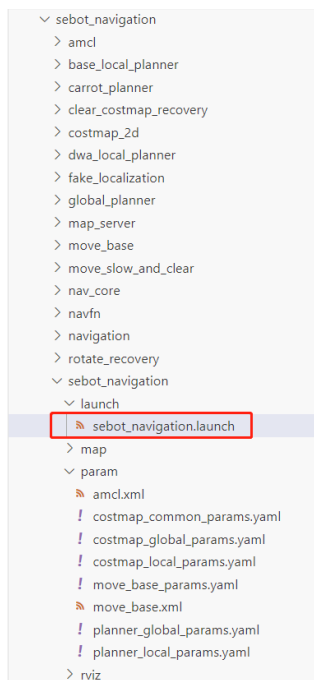


图 4-1 自主导航功能启动文件

详细的导航功能启动文件内容如下：

```
src > sebot_navigation > sebot_navigation > launch > sebot_navigation.launch
1  <!-- 导航 -->
2  <launch>
3    <!-- 启动机器人的控制节点 -->
4    <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
5      <!-- 是否发布传感器话题 value: true发布 false不发布 -->
6      <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
7      <param name="odomEnable" type="bool" value="true"/> <!-- 里程计使能 -->
8      <param name="imuEnable" type="bool" value="true"/> <!-- imu使能 -->
9    </node>
10
11    <!-- 发布机器人静态坐标 -->
12    <node pkg="sebot_robot" type="sebot_transform" name="sebot_transform" respawn="true"/>
13
14    <!-- 启动融合里程计 -->
15    <include file = "${find robot_pose_ekf}/launch/robot_pose_ekf.launch"/>
16
17    <!-- 启动激光雷达 -->
18    <include file = "${find rplidar_ros}/launch/rplidar.launch" />
19
20    <!-- 加载SLAM地图 -->
21    <node name="map_server" pkg="map_server" type="map_server" args="${find sebot_navigation)/map/map.yaml" respawn="true">
22      <param name="map" value="/map"/>
23    </node>
24
25    <!-- 启动蒙特卡洛定位算法 -->
26    <include file="${find sebot_navigation)/param/amcl.xml"/>
27
28    <!-- 启动move_base路径规划 -->
29    <include file = "${find sebot_navigation)/param/move_base.xml" />
30
31    <!-- 启动仿真监控界面 -->
32    <node pkg="rviz" type="rviz" name="rviz" args="-d ${find sebot_navigation)/rviz/navigation.rviz"/>
33
34  </launch>
```

图 4-2 导航功能启动文件内容

在导航功能的 launch 启动文件中，分别启动了相关的传感器，调用了我们之前学习过的坐标变换和 SLAM 建图后保存的地图文件，以及最为重要的对路径规划和定位算法的参数配置。其中加载的地图文件即我们前一次实训学习过的地图的配置文件。如果要使用其他地图，需要先确保此文件夹内包含对应的地图图片文件和地图配置文件，然后直接修改为需要加载的地图配置文件名称即可。

```
<!-- 加载SLAM地图 -->
<node name="map_server" pkg="map_server" type="map_server" args="$(find sebot_navigation)/map/map.yaml" respawn="true">
  <param name="map" value="/map"/>
</node>
```

图 4-3 启动文件中的地图配置信息

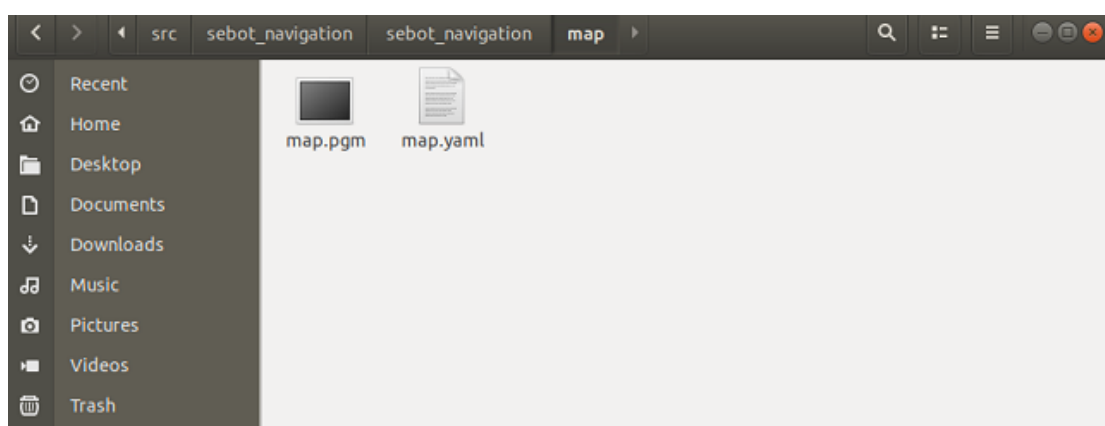


图 4-4 地图所在路径

步骤 2：自主导航 AMCL 定位参数配置

在 sebot_navigation.launch 启动文件中，可以看到加载了定位算法的相关配置文件。

```
<!-- 启动蒙特卡洛定位算法 -->
<include file="$(find sebot_navigation)/param/amcl.xml"/>
```

图 4-5 启动蒙特卡洛算法

智能机器人采用的定位算法是 AMCL (Adaptive Monte Carlo Localization, 自适应蒙特卡洛定位)，此算法是一种基于蒙特卡罗方法的粒子滤波算法，经常用于机器人的定位和导航。它通过将机器人在地图上的位置表示为一组粒子，每个粒子代表机器人在地图上的一个可能的位置，来进行定位和导航。

AMCL 算法的核心在于粒子的重采样。在每一次机器人运动和传感器测量后，所有粒子的权重都会发生变化，权重高的粒子将更有可能被保留下来，权重低的粒子将更有可能被删除。粒子的权重是根据机器人运动和传感器测量的数据计算得出的，因此重采样后的粒子集合更加接近机器人在地图上的真实位置。

在 AMCL 算法中，还有一个重要的概念是粒子滤波器。粒子滤波器是一种基于概率的算法，通过对机器人的运动和传感器测量进行概率计算，来估计机器人在地图上的位置。在 AMCL 算法中，粒子滤波器用于计算每个粒子的权重，并通过对粒子进行重采样来更新粒子的位置和姿态。

在智能机器人中，已经内置好了 AMCL 定位算法的源码，我们要掌握的是其具体的配置和启动内容。AMCL 定位算法的配置文件路径 `src\sebot_navigation\sebot_navigation\param` 如下。

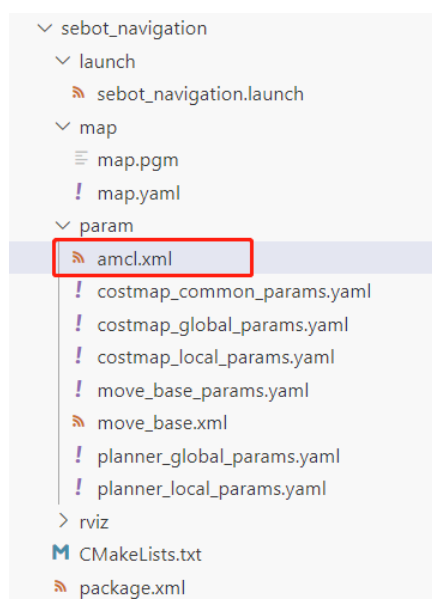


图 4-6 定位算法配置文件

此配置文件与前面讲解的建图配置文件类似，通过设置具体的参数数值并将参数传入到智能机器人已经内置好的 AMCL 节点中。文件中先是配置了基础的相关信息，然后配置 AMCL 算法中需要使用到的滤波器参数、初始例子束参数、激光雷达参数以及里程计模型参数。其中前两者是算法计算时需要的参数，后两者则是通过读取传感器和自行设置的。

1、话题及滤波器参数设置：包含里程计 TF 接收设置、机器人初始化姿态坐标、全局静态地图话题、激光雷达话题及滤波器中最大最小粒子数设置（400——5000）、滤波更新次数等。

```
<arg name="odom_frame_id"
default="odom_combined"/>          <!-- 接收里程计 TF -->
<arg name="map" default="map"/>      <!-- 依赖的全局静态地图话题 -->
<arg name="init_pose_x" default="0.0"/> <!-- 机器人初始化姿态：仿真与实际差异 -->
<arg name="init_pose_y" default="0.0"/> <!-- 机器人初始化姿态：仿真与实际差异 -->
```

```

<arg name="init_pose_a" default="0.0"/> <!-- 机器人初始化姿态：仿真与实际
差异 -->

<!-- AMCL -->
<node pkg="amcl" type="amcl" name="amcl">
  <remap from="scan" to="scan"/> <!-- 激光雷达话题 -->
  <remap from="map" to="$(arg map)" /> <!-- 依赖的全局静态地图话题 -->
  <param name="odom_frame_id" value="$(arg odom_frame_id)"/>
  <!-- 设置 AMCL 滤波器的参数 -->
  <param name="min_particles" value="400"/> <!-- 滤波器中的最少粒子
数，值越大定位效果越好，但是增加 CPU 消耗 -->
  <param name="max_particles" value="5000"/> <!-- 滤波器中最多粒子数，
上限值，因为太多的粒子数会导致系统资源消耗过多 -->
  <param name="kld_err" value="0.01"/> <!-- 真实分布与估计分布之间的最大
误差 -->
  <param name="kld_z" value="0.99"/> <!-- 上标准分位数 (1-p)，其中 p
是估计分布上误差小于 kld_err 的概率，默认 0.99 -->
  <param name="update_min_d" value="0.05"/> <!-- 执行滤波更新前平移运
动的距离，对于里程计模型有影响，这个值建议不大于机器人半径 -->
  <param name="update_min_a" value="0.05"/> <!-- 执行滤波更新前旋转的
角度：rad -->
  <param name="resample_interval" value="2"/> <!-- 在重采样前需要滤
波更新的次数 -->
  <param name="transform_tolerance" value="0.5"/> <!-- tf 变换发布推
迟的时间：s -->
  <param name="recovery_alpha_slow" value="0.001"/> <!-- 慢速的平均权重
滤波的指数衰减频率 -->
  <param name="recovery_alpha_fast" value="0.1"/> <!-- 快速的平均权重
滤波的指数衰减频率 -->

```

2、初始例子束参数设置：

```

<!-- 以初始粒子束估计机器人姿态 -->
<param name="initial_pose_x" value="$(arg
init_pose_x)"/> <!-- 初始化粒子滤波器的平均值和协方差，估计机器人的初始位姿
-->
<param name="initial_pose_y" value="$(arg init_pose_y)"/>
<param name="initial_pose_a" value="$(arg init_pose_a)"/> <!--
初始位姿均值 (yaw) -->
<param name="initial_cov_xx" value="0.25"/> <!-- 初始位姿协方差
(x*x) -->
<param name="initial_cov_yy" value="0.25"/>
<param name="initial_cov_aa" value="0.068538917"/> <!--  $\pi/12 * \pi/12$ 
-->
<param name="gui_publish_rate" value="-1.0"/> <!-- 扫描和路
径发布到可视化软件的最大频率：Hz，-1.0 失能此功能 -->

```

```

    <param name="save_pose_rate" value="0.5"/> <!-- 存储上一次估计的位姿和协方差到参数服务器的最大速率: Hz, -->
    <param name="use_map_topic" value="false"/> <!-- true:实时建图并发布map, false:通过map server 获取地图 -->
    <param name="first_map_only" value="false"/> <!-- 当设置为true时, AMCL 将仅仅使用订阅的第一个地图, 否则当收到新的地图后更新-->

```

3、激光雷达的参数设置, 包括最大、最小扫描范围, 地图上障碍物膨胀的最大距离等。

```

<!-- 设置激光雷达的参数 -->
<param name="laser_min_range" value="0.2"/> <!-- 最小扫描范围, 参数设置为-1.0 时, 将会使用雷达上报的最小扫描范围 -->
<param name="laser_max_range" value="3.5"/> <!-- 最大扫描范围, 参数设置为-1.0 时, 将会使用雷达上报的最小扫描范围 -->
<param name="laser_max_beams" value="60"/> <!-- 更新滤波器时, 每次扫描中多少个等间距的光束被使用, 减小计算量, 太小会造成信息量少定位不准 -->
<param name="laser_z_hit" value="0.95"/> <!-- 模型的 z_hit 部分的混合权值 -->
<param name="laser_z_short" value="0.1"/>
<param name="laser_z_max" value="0.05"/>
<param name="laser_z_rand" value="0.05"/>
<param name="laser_sigma_hit" value="0.2"/>
<param name="laser_lambda_short" value="0.1"/>
<param name="laser_likelihoood_max_dist" value="2.0"/> <!-- 地图上做障碍物膨胀的最大距离: m -->
<param name="laser_model_type" value="likelihood_field"/> <!-- 激光模型类型选择 -->

```

4、里程计模型参数设置, 包括里程计模型选择, 运动旋转或平移的期望噪声、基坐标系等。

```

<!-- 设置里程计模型参数 -->
<param name="odom_model_type" value="diff"/> <!-- odom 模型选择-->
<param name="odom_alpha1" value="0.03"/> <!-- 指定由机器人运动部分的旋转分量估计的里程计旋转的期望噪声 -->
<param name="odom_alpha2" value="0.03"/> <!-- 机器人运动部分的平移分量估计的里程计旋转的期望噪声 -->
<param name="odom_alpha3" value="0.03"/> <!-- 里程计平移的期望噪声, 如果自认为机器人的里程计信息比较准确那么可以减少该值 -->
<param name="odom_alpha4" value="0.03"/> <!-- 机器人运动部分的旋转分量估计的里程计平移的期望噪声, 设置前 4 个值越大说明里程计的误差越大 -->
<param name="odom_alpha5" value="0.0"/> <!-- 平移相关的噪声参数, 仅适用于全向移动机器人 (omni) -->
<param name="base_frame_id" value="base_footprint"/> <!-- 机器人的基坐标系 -->

```

```

<param name="global_frame_id" value="/map"/> <!-- 由定位系统发布的
坐标系名称 -->
<param name="tf_broadcast" value="true"/> <!-- 设置为false阻止amcl
发布全局坐标系和里程计坐标系之间的tf变换 -->

```

如果需要调整此配置文件的参数，可结合 AMCL 算法自行修改，修改时建议复制并重命名配置文件，并在功能启动 launch 文件中修改相应的文件路径。

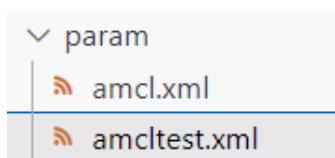


图 4-7 复制并重命名配置文件

```

25 <!-- 启动蒙特卡洛定位算法 -->
26 <include file="$(find sebot_navigation)/param/amcltest.xml"/>

```

图 4-8 修改功能启动文件中的相应路径

步骤 3：自主导航路径规划参数配置

路径规划的配置文件中除了设置一些具体的参数外，还引用了其他文件中的一些参数。`<rosparam>` 标签的含义即引用多个参数，简单理解为将另外一个配置文件中的参数进行引用并且设置权限即可。路径规划的配置文件 `move_base.xml` 路径为 `src\sebot_navigation\sebot_navigation\param`，如下图。

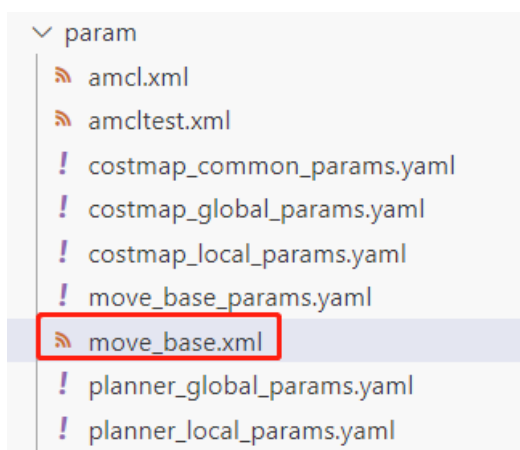


图 4-9 路径规划的配置文件

在学习此配置文件的功能逻辑之前，首先需要了解代价地图和路径规划的相关内容，`move_base.xml` 配置文件中加载二者的部分如下：

```

<!-- 配置代价地图的公共属性 -->
<roscpp file="$(find sebot_navigation)/param/costmap_common_params.yaml" command="load" ns="global_costmap"/>
<roscpp file="$(find sebot_navigation)/param/costmap_common_params.yaml" command="load" ns="local_costmap"/>

<!-- 配置全局代价地图和路径规划器 -->
<roscpp file="$(find sebot_navigation)/param/costmap_global_params.yaml" command="load" />
<roscpp file="$(find sebot_navigation)/param/planner_global_params.yaml" command="load" />

<!-- 配置局部代价地图和路径规划器 -->
<roscpp file="$(find sebot_navigation)/param/costmap_local_params.yaml" command="load" />
<roscpp file="$(find sebot_navigation)/param/planner_local_params.yaml" command="load" />

<!-- 配置路径规划器 (启动move_base) 软件包 -->
<roscpp file="$(find sebot_navigation)/param/move_base_params.yaml" command="load" />

```

图 4-10 配置各项参数文件

下面详细说明具体的各项配置。

(1) 代价地图配置

代价地图是 ROS 中用于导航的一个关键组件，特别是在全局路径规划中。它是一种在机器人导航过程中，用于生成机器人周围环境地图的方法。

代价地图是一个二维网格地图，其中每个单元格表示机器人可以占据的空间或障碍物。它根据机器人传感器收集的信息构建而成，通常使用激光雷达或里程计数据作为输入。在构建代价地图时，算法将传感器数据与预先存储的地图数据进行比较，以确定机器人的位置和姿态。然后，基于机器人的姿态和传感器数据，算法将计算每个单元格的代价值，表示机器人从当前位置到该单元格的代价，例如移动距离、时间、风险等因素。

智能机器人中应用了两种代价地图，一种是全局代价地图，主要是基于加载的地图文件进行代价计算；另一种是局部代价地图，主要是基于机器人当前位置附近的障碍物情况进行代价计算。全局代价地图用于全局路径规划，可以用于计算从起点到终点的导航路线；局部代价地图则用于局部路径规划，可以灵活应对导航路线上出现障碍物的情况。两种代价地图需要使用一些共同的配置即通用配置文件。

相关的代价地图配置文件名称及含义如下：

costmap_common_params.yaml: 代价地图通用配置文件

costmap_global_params.yaml: 全局代价地图配置文件

costmap_local_params.yaml: 局部代价地图配置文件

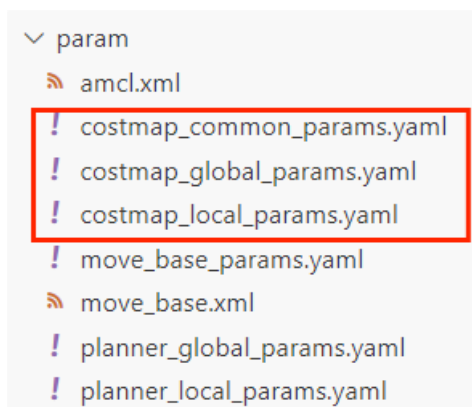


图 4-11 代价地图配置文件

打开配置文件即可查看具体的配置信息，文件中我们需要知道并且可以观察到的是通过配置文件可以生成三个层，分别是静态层、障碍层以及膨胀层。静态层即根据加载的地图文件生成的基本层，主要用于生成地图信息；障碍层是根据障碍物的信息在地图上生成的障碍物标示；膨胀层则是根据人为设定，用于防止机器人撞上障碍物的层，数值一般不小于机器人的底盘半径，因此此数值为底盘中心到障碍物的距离。

在全局代价地图和局部代价地图中还会有具体的差别，全局代价地图基于加载的地图文件，所以变化较少；而局部代价地图则基于实时接收的传感器数据，因此会实时刷新。

这些属性可以在启动导航功能后开启的 Rviz 软件上进行显示，下图即加载的示例地图。

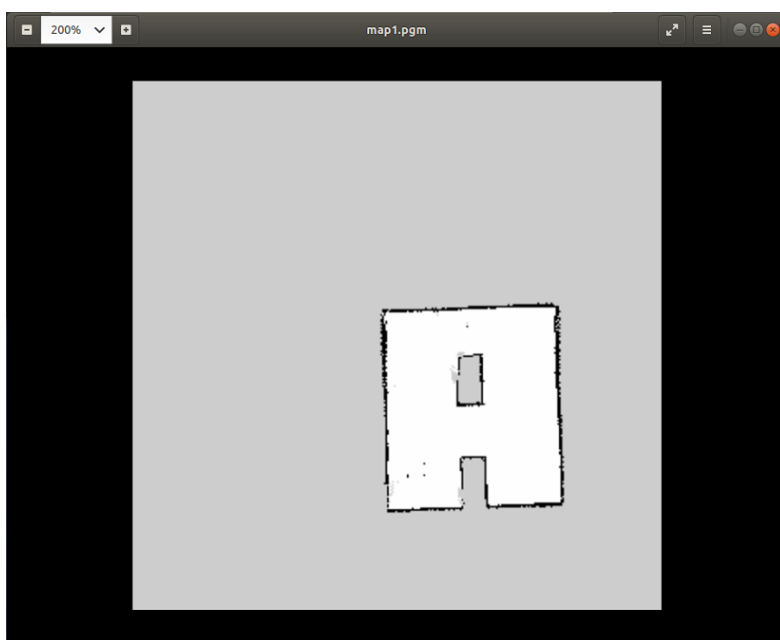


图 4-12 加载的地图形状

执行自主导航功能后，Rviz 中会出现根据加载的地图和实际的周围情况生成代价地图。彩色地图部分为全局代价地图，其中灰色部分即安全的可行驶路段，浅蓝色部分为障碍物，

粉色和紫色则是不同代价值的膨胀层。绿色多边形线条即机器人的轮廓，机器人周围的黑色方形阴影部分及灰色气团状区域为局部代价地图。两个代价地图之间会互相影响，以此共同实现导航功能。

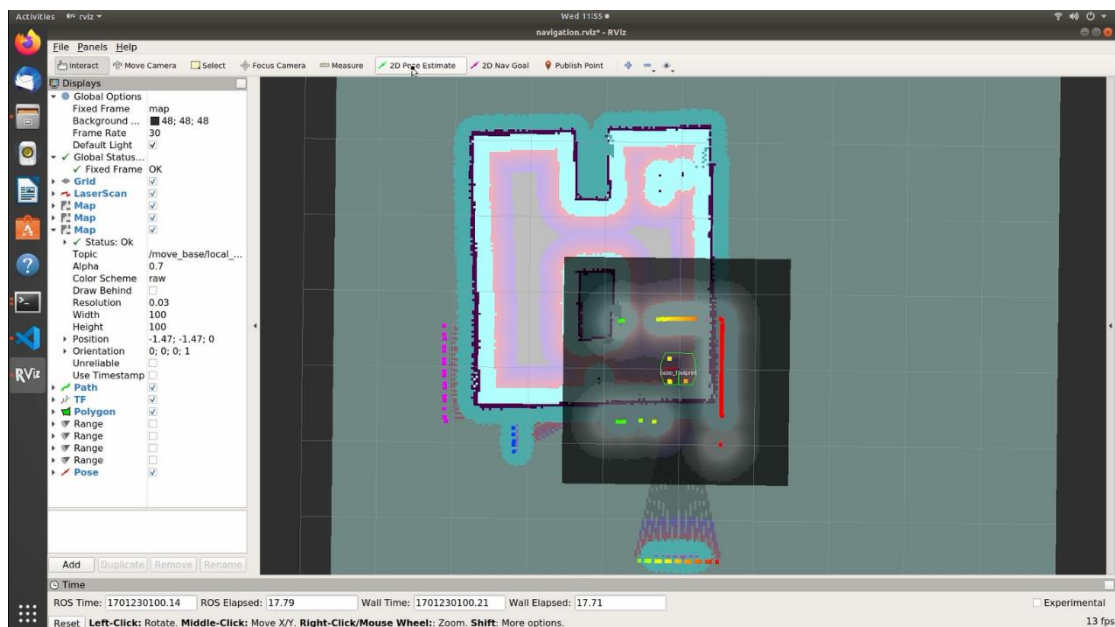


图 4-13 生成的代价地图

(2) 路径规划器配置

智能机器人的路径规划配置文件路径为 `src\sebot_navigation\sebot_navigation\param`，如下图所示。

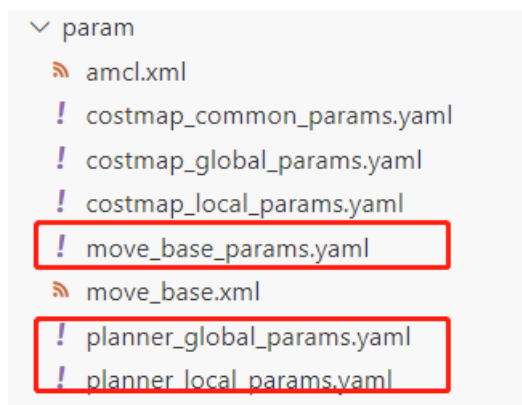


图 4-14 路径规划配置文件

相关的路径规划配置文件名称及含义如下：

`planner_global_params.yaml`：全局路径规划配置

`planner_local_params.yaml`：局部路径规划配置

`move_base_params.yaml`：路径规划器软件包配置

全局规划是从出发点到目的地之间的纯几何路径规划,与时间序列和智能机器人的动力学无关。它主要关注的是在给定的起点和终点之间确定一条最优路径,而不会考虑机器人实际移动过程中的细节。在全局规划中,通常使用各种算法(如 Dijkstra 算法或 A*算法)来搜索和优化路径。

局部规划是在全局规划的基础上,考虑机器人的动力学特性和环境中的动态障碍物,即在机器人实际移动过程中考虑如何避免碰撞、绕过障碍物,并适应环境中的动态变化。

代价地图信息以及两种路径规划器的相关参数最终全部都会加载到路径规划器软件包中进行计算,最后得出机器人的实际控制信息。

路径规划器软件包的详细配置内容如下:

```
src > sebot_navigation > sebot_navigation > param > ! move_base_params.yaml
1  # move_base软件包配置
2  # 选择全局路径的规划插件:
3  # — navfn:ROS中比较旧的代码实现了dijkstra和A*全局规划算法。
4  # — global_planner:重新实现了Dijkstra和A*全局规划算法,可以看作Navfn的改进版。
5  # — parrot_planner:一种简单的算法实现全局路径规划算法。
6  # 选择局部路径的规划插件:
7  # — base_local_planner:实现了Trajectory Rollout和DWA两种局部规划算法。
8  # — dwa_local_planner:实现了DWA局部规划算法,可以看作是base_local_planner的改进版本
9
10 shutdown_costmaps: false # 当move_base在不活动状态时,是否关掉代价地图
11 controller_frequency: 3.0 # 向底盘控制移动话题cmd_vel发送命令的频率
12 controller_patience: 3.0 # 在空间清理操作执行前,控制器花多长时间等有效控制下发
13 planner_frequency: 1.0 # 全局规划操作的执行频率,如果设置为0.0,则全局规划器仅在接收到新的目标点或者局部规划器报告路径堵塞时才会重新执行规划操作。
14 planner_patience: 3.0 # 在空间清理操作执行前,留给规划器多长时间来找出一条有效规划
15 oscillation_timeout: 5.0 # 执行修复机制前,允许振荡的时长
16 oscillation_distance: 0.3 # 来回运动在多大距离以上不会被认为是振荡
17 base_local_planner: "dwa_local_planner/DWAPlannerROS" #指定用于move_base的局部规划器插件名称
18 base_global_planner: "global_planner/GlobalPlanner" #指定用于move_base的全局规划器插件名称
19
20
21 #旋转恢复行为参数
22 ~ TrajectoryPlannerROS:
23   yaw_goal_tolerance: 0.05 # 0.05 #控制器在实现其目标时偏差/旋转的弧度公差
24   acc_lim_theta: 3.2 # 3.2 #机器人的旋转加速度限制,单位为弧度/秒^2
25   max_vel_theta: 0.3 # 1.0 #机器人允许的最大旋转速度,单位为弧度/秒
26   min_in_place_vel_theta: 0.1 #0.4 #以弧度/秒执行就地旋转时基座允许的最小旋转速度
27
28
```

图 4-15 路径规划器软件包配置

(3) 整体逻辑

理解了代价地图和路径规划的功能后,我们再仔细查看路径规划功能配置文件 move_base.xml 的详细内容。

配置文件中,首先是设定全局坐标系,然后是在机器人移动的节点中配置了代价地图和两种路径规划器的相关信息,接着调用路径规划器软件包进行路径规划的计算,最后设置坐标系并发布话题消息控制机器人进行移动。

详细内容如下:

```
<launch>
  <arg name="odom_frame_id" default="odom_combined"/>      <!-- 代
价地图中的全局坐标系 -->
  <arg name="global_frame_id" default="map"/>
  <arg name="map_topic" default="/map"/>
```

```

<!-- 启动move_base 机器人移动节点 -->
<node pkg="move_base" type="move_base" name="move_base"
output="screen" respawn="true" >
  <!-- 配置代价地图的公共属性 -->
  <rosparam file="$(find
sebot_navigation)/param/costmap_common_params.yaml" command="load"
ns="global_costmap"/>
  <rosparam file="$(find
sebot_navigation)/param/costmap_common_params.yaml" command="load"
ns="local_costmap"/>

  <!-- 配置全局代价地图和路径规划器 -->
  <rosparam file="$(find
sebot_navigation)/param/costmap_global_params.yaml" command="load" />
  <rosparam file="$(find
sebot_navigation)/param/planner_global_params.yaml" command="load" />

  <!-- 配置局部代价地图和路径规划器 -->
  <rosparam file="$(find
sebot_navigation)/param/costmap_local_params.yaml" command="load" />
  <rosparam file="$(find
sebot_navigation)/param/planner_local_params.yaml" command="load" />

  <!-- 配置路径规划器（启动move_base）软件包 -->
  <rosparam file="$(find
sebot_navigation)/param/move_base_params.yaml" command="load" />

  <!-- 设置代价地图中的坐标系 -->
  <param name="global_costmap/global_frame" value="$(arg
global_frame_id)"/> <!-- 指定全局代价地图的全局坐标系 -->
  <param name="global_costmap/robot_base_frame"
value="base_footprint"/> <!-- 指定全局代价地图的机器人坐标系
-->
  <param name="local_costmap/global_frame" value="$(arg
odom_frame_id)"/> <!-- 指定局部代价地图的全局坐标系-->
  <param
name="local_costmap/robot_base_frame" value="base_footprint"/>
  <!-- 指定局部代价地图的机器人坐标系 -->
  <param name="DWAPlannerROS/global_frame_id" value="$(arg
odom_frame_id)"/>
  <param
name="base_local_planner" value="dwa_local_planner/DWAPlan
nerROS"/>

```

```

        <remap from="/cmd_vel" to="/cmd_vel"/>           <!-- cmd_vel 速控
话题 -->
        <remap from="/odom" to="/odom_combined"/>       <!-- odom 里程计话
题 -->
        <remap from="/map" to="$(arg map_topic)"/>       <!-- 地图话题 -->
    </node>
</launch>

```

步骤 4：自主导航实操

运行功能前先确保地图文件与真实环境一致，并且地图图片与配置文件一致。如果需要修改地图，则在 `sebot_ros_kits/src/sebot_navigation/sebot_navigation/map` 文件夹下确保地图文件存在，然后在定位导航的启动文件 `sebot_navigation.launch` 中修改对应的文件名。

此步骤建议直接使用上一个实验建图功能完成后保存的地图文件，使用建图功能完成后保存的地图文件时，只需要修改定位导航启动文件中的文件名使其对应即可。

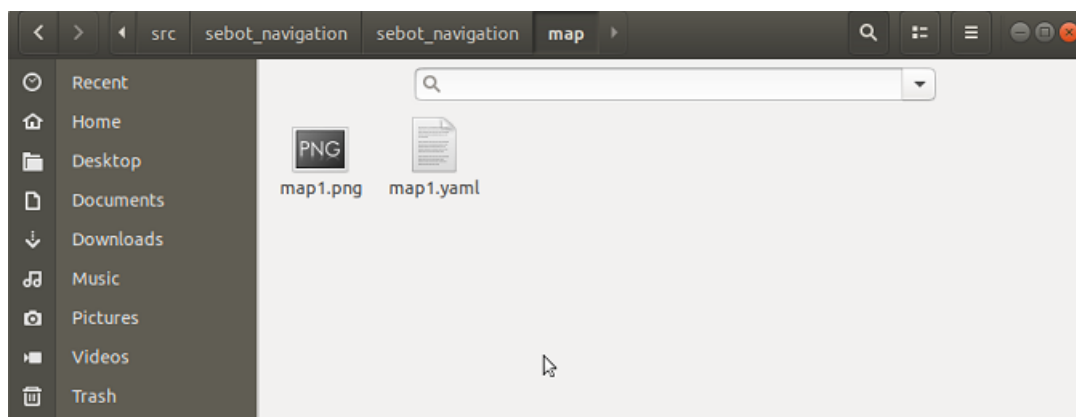


图 4-16 确保地图文件正确存在

```

<!-- 加载SLAM地图 -->
<node name="map_server" pkg="map_server" type="map_server" args="$(find sebot_navigation)/map/map1.yaml" respawn="true">
  <param name="map" value="/map"/>
</node>

```

图 4-17 修改加载的地图文件名

设置完成后，基于智能机器人打开终端并输入以下命令即可开启自主定位与导航的功能。

```
roslaunch sebot_navigation sebot_navigation.launch
```

功能开启后系统会自动打开 **Rviz** 软件，等待软件加载完代价地图以及相关坐标系信息。

机器人起点位置及朝向设置。选择上方的“2D Pose Estimate”按钮，设置机器人初始的起点位姿信息。选择此功能后根据机器人在真实环境中的位置对应到软件中的相应位置，单击鼠标并拖动鼠标，即可设置机器人的起点位置和姿态（即朝向）。起点设置完成后机器人的位置会发生改变。

目标点位置设置。单击上方的“2D Nav Goal”按钮，此按钮即设定机器人导航的终点信息，设置方式与设置起点相同，设置完终点后，机器人即可自主导航。

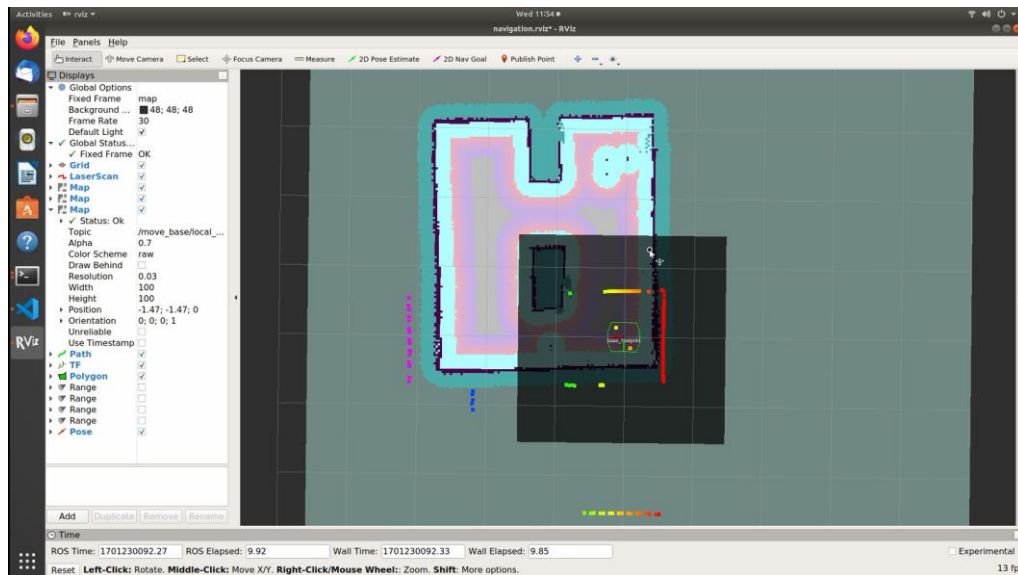


图 4-18 地图加载完成

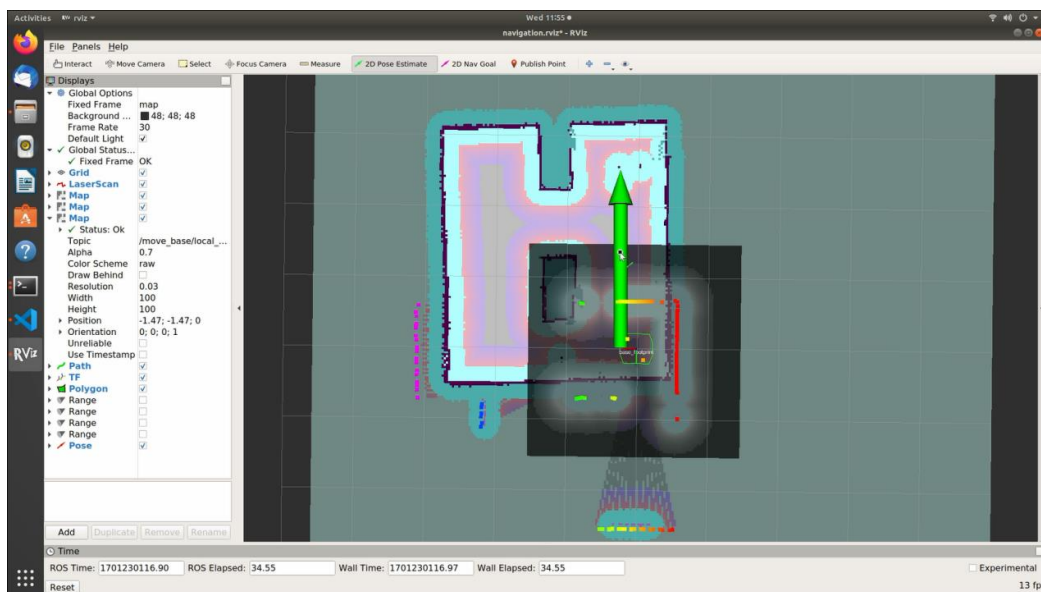


图 4-19 设置起点，绿色箭头

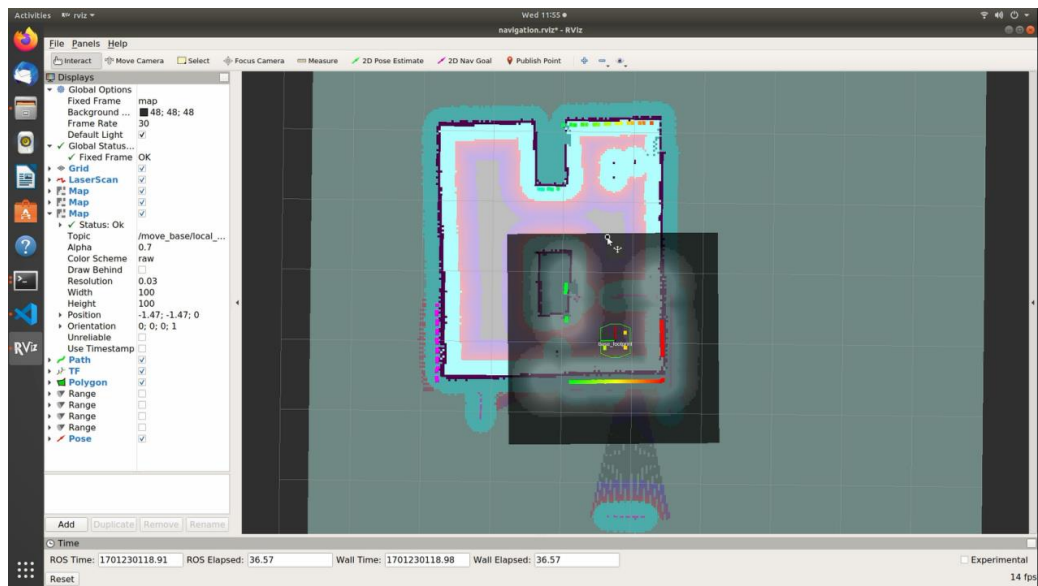


图 4-20 起点设置完成

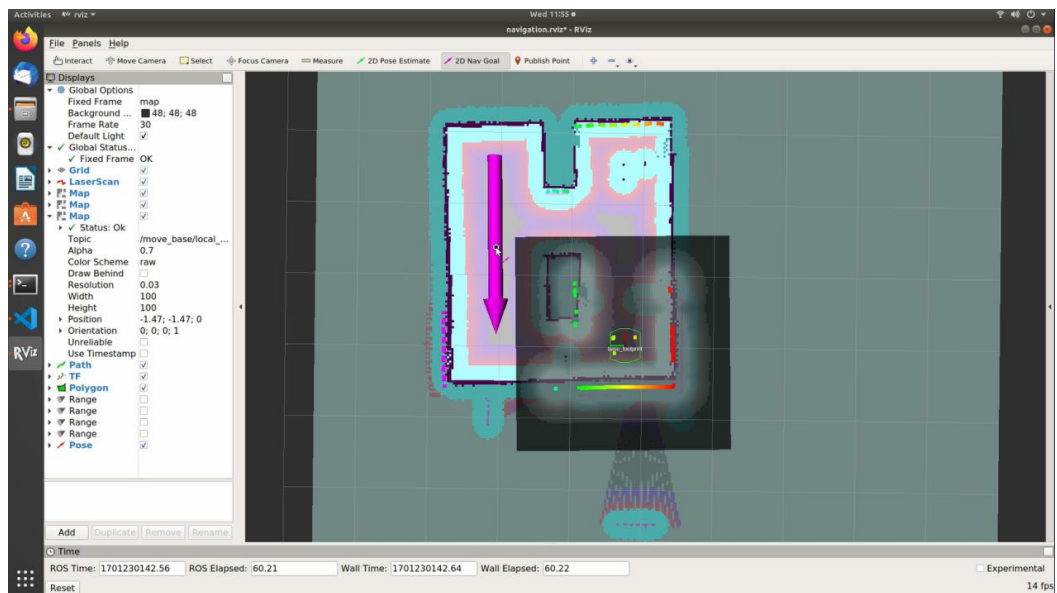


图 4-21 设置终点，紫色箭头

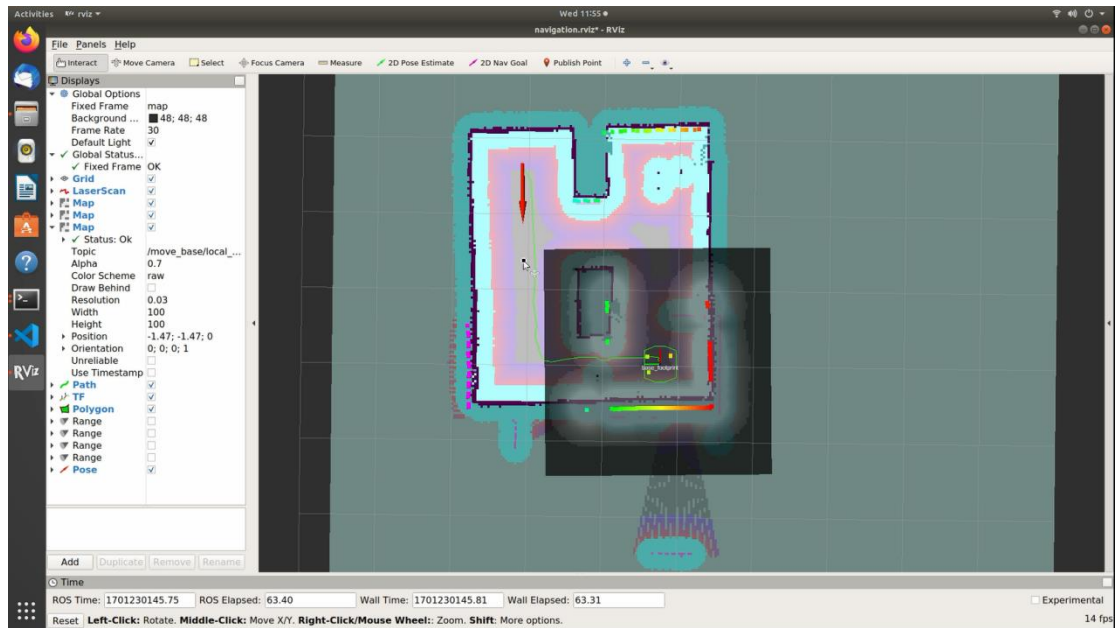


图 4-22 终点设置完成，出现路径规划

步骤 5：还原启动文件配置

测试时如果需要调整参数并观察运行结果，建议采用复制并重命名配置文件的方式，避免对原工程文件造成无法复原的影响。在完成学习和测试后，将相关的配置文件中相应的路径还原，以免影响后续其他功能的使用。